

HÉBERGEMENT D'APPLICATION

Déployer Bookaru

Du dépôt GitHub au NAS, puis à Internet en HTTPS, avec mises à jour automatiques

Axel Royer

Novembre 2026

1. Objectif et vue d'ensemble

Bookaru est une application web de suivi de lecture (PWA), développée par mes soins et conservée sur GitHub. Elle n'était accessible que par GitHub Pages. L'objectif de ce document est de décrire **comment elle a été déposée sur mon NAS, puis mise en ligne sur Internet**, avec trois exigences :

- l'héberger moi-même, en gardant **GitHub comme source de vérité** du code ;
- la rendre accessible **publiquement en HTTPS**, sans ouvrir aucun port sur la box ;
- que toute modification poussée sur GitHub soit **mise en ligne automatiquement**.

Le déploiement se fait en quatre étapes :

Étape	Objet
A	Déposer Bookaru sur le NAS et la servir (clone git + serveur web Caddy)
B	La mettre sur Internet (domaine + Cloudflare Tunnel)
C	Durcir l'accès (l'application ne passe plus que par le tunnel)
D	La mettre à jour (manuellement, puis automatiquement)

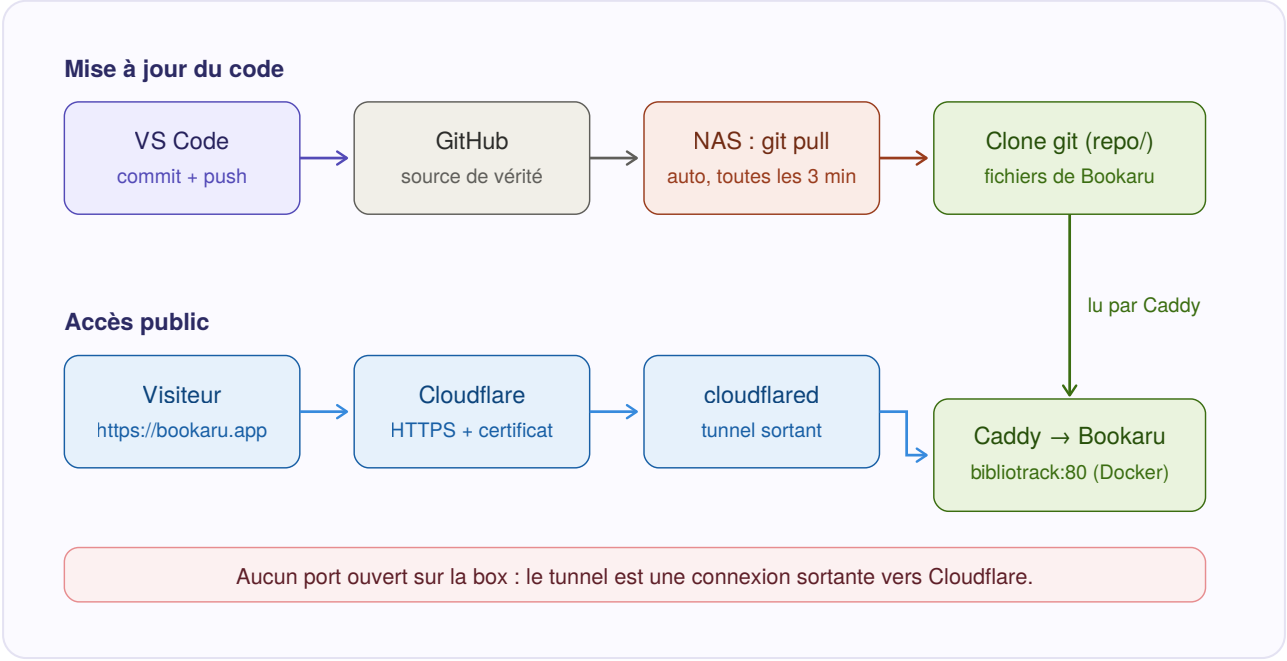


Figure : en haut, le chemin d'une modification du code ; en bas, le chemin d'un visiteur.

Les briques utilisées :

Brique	Rôle
Git / GitHub	Stockage et versionnement du code ; le NAS n'en détient qu'une copie
Caddy	Serveur web qui distribue les fichiers de l'application
Docker / Portainer	Exécution des services en conteneurs, gérés par des « stacks »
Cloudflare (domaine + Tunnel)	Nom de domaine, HTTPS public, tunnel sans port ouvert
`cloudflared`	Connecteur du tunnel, exécuté en conteneur sur le NAS

2. L'application à déployer

Bookaru n'est composée que de **fichiers statiques** (HTML, CSS, JavaScript, images, manifeste). Il n'y a donc **rien à exécuter côté serveur** : un simple serveur web qui distribue ces fichiers suffit. Le code, public, est sur `github.com/Relyzim/app_biblio_nini` :

```
app_biblio_nini/
├── index.html      structure de la page
├── style.css       mise en forme
├── app.js          logique de l'application
├── sw.js           service worker (cache et hors-ligne)
├── manifest.json   métadonnées PWA
├── icons/          icônes d'application
└── LISEZ-MOI.md
```

Deux points conditionnent le déploiement :

- `index.html` **est à la racine** du dépôt : le serveur web pourra le servir directement.
- **Le HTTPS est indispensable** à l'installation de la PWA et à son mode hors-ligne : le service worker et l'accès à la caméra (scan de codes-barres) sont refusés par les navigateurs hors contexte sécurisé. C'est la raison de l'étape B.

3. Étape A — Déposer Bookaru sur le NAS

3.1 Principe

GitHub reste la source de vérité : je ne copie jamais de fichiers à la main sur le serveur. Le NAS récupère une **copie du dépôt** (un clone git) et Caddy sert directement ce dossier. Mettre à jour l'application revient alors à récupérer les dernières modifications, sans rien recopier.

3.2 Arborescence sur le NAS

Chaque application a son propre dossier dans l'espace des données des conteneurs :

```
appdata/bibliotrack/
├── repo/        clone git du dépôt GitHub (contient aussi le dossier caché .git)
└── Caddyfile    configuration du serveur web
```

(Le dossier porte le nom technique `bibliotrack`, nom de l'application avant son changement en Bookaru.)

3.3 Cloner le dépôt

Le dépôt étant public, un clone HTTPS suffit, sans identifiant. Les commandes sont lancées en SSH sur le NAS ; le chemin des données est stocké dans une variable pour éviter de retaper l'identifiant du disque :

```
APPDATA=/srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata
mkdir -p $APPDATA/bibliotrack
cd $APPDATA/bibliotrack
git clone https://github.com/Relyzim/app_biblio_nini.git repo
ls $APPDATA/bibliotrack/repo
```

Le dernier argument (`repo`) impose le nom du dossier local. Le `ls` doit afficher les fichiers du dépôt, dont `index.html` .

Il s'agit d'un flux **sortant** (le NAS va chercher le code) : le HTTPS de `git clone` n'a aucun rapport avec le protocole utilisé ensuite pour servir l'application.

3.4 Configurer le serveur web (Caddyfile)

Fichier `appdata/bibliotrack/Caddyfile` :

```
:80 {
    root * /srv/repo
    encode gzip

    @git path /.git/*
    respond @git 404

    try_files {path} /index.html
    file_server
}
```

Ligne	Rôle
`:80`	Caddy écoute sur le port 80 **à l'intérieur du conteneur**
`root * /srv/repo`	Dossier des fichiers, tel que vu depuis le conteneur
`encode gzip`	Compresse les réponses
`@git … respond 404`	**Bloque l'accès au dossier** `.git` : sans cela, l'historique complet du code serait téléchargeable
`try_files {path} /index.html`	Si l'URL ne correspond à aucun fichier, renvoie `index.html` : règle nécessaire au bon fonctionnement d'une PWA
`file_server`	Active la distribution des fichiers statiques

3.5 Lancer le conteneur

Stack Portainer (nom : `bibliotrack`) :

```
services:
  bibliotrack:
    image: caddy:2-alpine
    container_name: bibliotrack
    restart: unless-stopped
    ports:
      - "8095:80"
    volumes:
      - /srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata/bibliotrack/repo:/
srv/repo:ro
      - /srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata/bibliotrack/
Caddyfile:/etc/caddy/Caddyfile:ro
```

- `restart: unless-stopped` : le conteneur redémarre seul après un redémarrage du NAS.
- `ports: "8095:80"` : le port 8095 du NAS est relié au port 80 du conteneur (celui du Caddyfile). Cette publication est **provisoire** ; elle sera retirée à l'étape C.
- Les deux volumes sont montés en **lecture seule** (`:ro`) : le conteneur ne peut modifier ni le code ni sa configuration.
- Dans une stack Portainer, la variable `$APPDATA` n'est pas interprétée : les chemins sont donc écrits en entier.

3.6 Vérification

Le conteneur doit être en état *running* dans Portainer. Depuis un appareil du réseau local, ouvrir `http://192.168.1.134:8095` : **Bookaru s'affiche**. L'application est hébergée sur le NAS.

À ce stade, l'accès se fait en **HTTP** sur le réseau privé : l'application s'affiche, mais le service worker ne s'enregistre pas et la PWA n'est pas installable. C'est ce que règle l'étape suivante.

---

4. Étape B — Mettre Bookaru sur Internet

4.1 Pourquoi un tunnel

Pour publier un service chez soi, la méthode classique consiste à ouvrir un port sur la box et à le rediriger vers le serveur : cela expose l'adresse IP du domicile et le serveur aux scans permanents d'Internet. J'ai retenu un **Cloudflare Tunnel** : un conteneur sur le NAS ouvre une connexion **sortante** vers Cloudflare, et c'est Cloudflare qui reçoit le trafic public pour le renvoyer dans ce tunnel. **Aucune redirection de port**, IP du domicile masquée, certificat HTTPS géré par Cloudflare.

4.2 Le nom de domaine

Le tunnel nommé exige un domaine dont Cloudflare gère le DNS. Le domaine **`bookaru.app`** a été enregistré **directement chez Cloudflare Registrar** (environ 14 $ par an) : il arrive déjà dans le compte, DNS géré automatiquement, sans manipulation de serveurs de noms. L'extension `.app` est sur la liste *HSTS preload* des navigateurs : l'accès en HTTP clair y est impossible, ce qui renforce la sécurité.

4.3 Activer Zero Trust

Les tunnels vivent dans la section **Zero Trust** (« Cloudflare One ») du tableau de bord. L'activation se fait en plan **Free**, gratuit et largement suffisant ici.

4.4 Créer le tunnel

Networking, Tunnels, Create a tunnel, type **Cloudflared**, nom `bookaru`. Cloudflare affiche un **jeton** (longue chaîne commençant par `eyJ`) dans les instructions d'installation. Dans la liste déroulante du système, choisir **Docker** : c'est le NAS qui exécute le connecteur, pas le poste de travail.

> **Le jeton est un secret.** Quiconque le possède peut faire tourner le tunnel. Il ne doit apparaître ni dans un fichier de configuration, ni dans un message. En cas de doute (exposition accidentelle), la page du tunnel permet de le **régénérer** (« Rotate token »), ce qui invalide l'ancien.

4.5 Déployer le connecteur `cloudflared`

Stack Portainer (nom : `cloudflared`) :

```
services:
  cloudflared:
    image: cloudflare/cloudflared:latest
    container_name: cloudflared
    restart: unless-stopped
    command: tunnel --no-autoupdate run
    environment:
      - TUNNEL_TOKEN=${CF_TUNNEL_TOKEN}
```

Le jeton n'est **pas écrit dans le YAML** : il est fourni par une variable d'environnement `CF_TUNNEL_TOKEN`, renseignée dans le champ dédié de Portainer au moment du déploiement.

Vérifications : le conteneur est en *running* (ses journaux affichent `Registered tunnel connection`) et, côté Cloudflare, le tunnel passe au statut **Healthy**.

4.6 Router le domaine vers Bookaru

Sur le tunnel, ajout d'une route **Published application** :

Champ	Valeur
Sous-domaine	*(vide)* pour servir `bookaru.app` directement
Domaine	`bookaru.app`
Service URL	`http://192.168.1.134:8095` (version initiale)

Le `http://` est volontaire : ce tronçon relie `cloudflared` à Caddy **à l'intérieur du NAS** et ne sort jamais de la machine. Le HTTPS public est assuré en amont par Cloudflare. Le champ exige un protocole : saisir `https://` ferait échouer la connexion, car Caddy n'écoute pas en HTTPS.

4.7 Vérification

Ouvrir `https://bookaru.app` : l'application s'affiche **en HTTPS, avec un certificat valide**, depuis n'importe où, y compris hors du réseau local (test en 4G). Le service worker s'enregistre et la **PWA devient installable**.

---

5. Étape C — Durcir l'accès

La chaîne fonctionne, mais le port 8095 reste ouvert **sur le réseau local**, en HTTP clair, et `cloudflared` passe par l'adresse du NAS. Le durcissement supprime cet accès : Bookaru ne doit plus être joignable **que par le tunnel**.

1. Créer un réseau Docker partagé :

```
docker network create bookaru-net
```

2. Rattacher les deux conteneurs à ce réseau, dans leur stack respective (en gardant le port 8095 à ce stade) :

```
    networks:
      - bookaru-net

networks:
  bookaru-net:
    external: true
```

Les deux conteneurs se voient alors par leur nom. Vérification :

```
docker network inspect bookaru-net --format '{{range .Containers}}{{.Name}} {{end}}'
```

La commande doit lister `bibliotrack` et `cloudflared`.

3. Basculer la route Cloudflare : Service URL passée de `http://192.168.1.134:8095` à `http://bibliotrack:80` (nom du conteneur Caddy, port interne). Test : `https://bookaru.app` doit répondre comme avant. Une erreur DNS fugace de `cloudflared` peut apparaître dans ses journaux au moment du changement : c'est un hoquet de transition normal.

4. Retirer la publication du port : supprimer les lignes `ports: - "8095:80"` de la stack de Caddy, puis redéployer.

Validation par test contrasté :

Test	Attendu	Résultat
`https://bookaru.app`	répond toujours	OK (le tunnel passe par le réseau Docker)
`http://192.168.1.134:8095`	injoignable	OK (refus de connexion)

Bookaru n'est plus accessible qu'à travers le tunnel : plus aucun accès en HTTP clair sur le réseau local.

---

6. Étape D — Mettre à jour Bookaru

6.1 Mise à jour manuelle

Récupérer les dernières modifications se fait **depuis l'intérieur du dépôt** (lancée ailleurs, la commande échoue avec « not a git repository ») :

```
cd /srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata/bibliotrack/repo
git pull
```

Caddy sert les fichiers **à chaud** : les nouveaux fichiers sont pris en compte immédiatement, sans redémarrer le conteneur. Le dépôt appartenant à root, la commande se lance avec les droits administrateur.

6.2 Forcer la mise à jour des appareils : `APP_VERSION`

Le service worker garde l'application en cache pour le mode hors-ligne : sans précaution, un appareil où Bookaru est installée continuerait à afficher l'ancienne version. Le fichier `sw.js` porte donc une constante :

```
const APP_VERSION = 'v1.02';
```

Les caches sont nommés d'après cette valeur : la **changer** force chaque appareil à supprimer l'ancien cache et à charger la nouvelle version. La règle :

- modification **visible** (HTML, CSS, JavaScript, icônes) : incrémenter `APP_VERSION` ;
- modification **sans effet** pour l'utilisateur (ex. le fichier `LISEZ-MOI.md`) : inutile.

Une version ne doit **jamais être réutilisée** : un appareil qui l'aurait déjà servie croirait n'avoir rien à mettre à jour.

6.3 Mise à jour automatique (déploiement continu)

Pour supprimer le `git pull` manuel et le risque de l'oublier, le NAS vérifie **toutes les 3 minutes** s'il y a du nouveau sur GitHub et met l'application à jour.

Choix : une tâche planifiée plutôt qu'un webhook GitHub. Un webhook serait instantané, mais obligerait à exposer un point d'entrée public supplémentaire à sécuriser. La tâche planifiée n'ouvre rien, et un délai de quelques minutes est sans conséquence pour cette application.

Script `auto-pull.sh` (dans `appdata/bibliotrack/`) :

```
#!/bin/bash
REPO="/srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata/bibliotrack/repo"
DISCORD_WEBHOOK="<URL_DU_WEBHOOK>"

cd "$REPO" || exit 1

# Récupère l'état distant sans fusionner
git fetch origin >/dev/null 2>&1
LOCAL=$(git rev-parse @)
REMOTE=$(git rev-parse @{u})

# Rien de nouveau : sortie silencieuse
if [ "$LOCAL" = "$REMOTE" ]; then
    exit 0
fi

# Mise à jour
if git pull --ff-only >/dev/null 2>&1; then
    MSG="Bookaru mis à jour automatiquement ($(git log -1 --pretty=%s))"
else
    MSG="Échec du pull auto Bookaru, intervention manuelle requise"
fi

curl -s -H "Content-Type: application/json" \
     -d "{\"content\": \"$MSG\"}" \
     "$DISCORD_WEBHOOK" >/dev/null 2>&1
```

- `git fetch` puis comparaison des empreintes : le script ne fait **rien** (et ne notifie pas) tant qu'il n'y a pas de nouveau commit.
- `git pull --ff-only` : n'applique que des avances propres, sans fusion hasardeuse, adapté à un dépôt que le NAS ne fait que suivre.
- Une **notification Discord** n'est envoyée que lorsqu'une mise à jour a eu lieu ou a échoué. L'URL du webhook est un secret : elle reste dans le script sur le NAS.

Le script est rendu exécutable (`chmod +x`), testé à la main, puis planifié dans l'interface de gestion du NAS (tâches planifiées) : utilisateur `root` (propriétaire du dépôt), fréquence `*/3 * * * *`, sans envoi de sortie par e-mail.

Le workflow final :

```
VS Code → commit → push GitHub ──► (le NAS interroge GitHub, tire et notifie, sous 3 min) ──► en ligne
```

---

7. Sécurité du déploiement

- **Aucun port ouvert sur la box.** Le tunnel est une connexion sortante ; l'adresse IP du domicile reste invisible.
- **HTTPS partout côté public.** Cloudflare gère le certificat et son renouvellement ; l'extension `.app` interdit l'accès en clair.
- **Rien d'autre que Bookaru n'est exposé.** Le tunnel ne publie que le nom de domaine qui lui est routé ; les autres services du NAS ne figurent pas dans le tunnel.

- **Après durcissement, plus d'accès HTTP en clair** sur le réseau local : l'application ne passe que par le tunnel, via un réseau Docker interne.
- **Dossier `.git` inaccessible** (réponse 404) et **volumes en lecture seule** : l'historique du code n'est pas exposé et le conteneur ne peut rien modifier.
- **Secrets protégés** : le jeton du tunnel est en variable d'environnement, jamais dans un fichier ; le webhook Discord reste dans le script sur le serveur.
- **Cloudflare devant la porte** : filtrage anti-DDoS et pare-feu applicatif de base. Contrepartie assumée : Cloudflare voit le trafic public déchiffré.

Bruit de fond. Dès sa mise en ligne, `bookaru.app` reçoit des visites automatiques (robots d'indexation, scanners de sécurité) : le tableau de bord Cloudflare affichait par exemple plus d'une centaine de « visiteurs uniques » et quelques signalements de sécurité alors que l'adresse n'avait été communiquée à personne. C'est normal pour tout site public, et sans conséquence ici : l'application est statique, sans compte ni donnée serveur, et ne donne aucun accès au reste du réseau.

---

8. Difficultés rencontrées

Symptôme	Cause	Résolution
`fatal: not a git repository`	`git pull` lancé hors du dépôt	Se placer dans `…/bibliotrack/repo` avant
Application à l'ancienne version sur un appareil	Cache du service worker	Incrémenter `APP_VERSION` dans `sw.js`
PWA non installable, pas de hors-ligne en HTTP	Service worker exige un contexte sécurisé	HTTPS public via le tunnel (étape B)
Menu du système sur « Windows » à la création du tunnel	Valeur par défaut de l'interface	Choisir **Docker**
`Invalid service URL format`	Protocole absent ou `https://` saisi	`http://` : Caddy sert en HTTP à l'intérieur du NAS
Erreur DNS fugace de `cloudflared`	Rechargement de la configuration de la route	Normale, disparaît une fois la route stabilisée
Jeton du tunnel affiché en clair lors d'un échange	Copie de la commande complète	Le considérer comme exposé et le régénérer (« Rotate token »)

---

9. Résultat et suites

Résultat. Bookaru est hébergée sur mon NAS, accessible en HTTPS à `https://bookaru.app` depuis n'importe où, installable en PWA, sans aucun port ouvert, et **mise à jour automatiquement** à chaque push sur GitHub.

Suites possibles :

- **Synchronisation multi-appareils** : aujourd'hui, les données restent sur chaque appareil. Une synchronisation exigerait des comptes utilisateurs et un back-end (authentification, base de données) : un chantier de développement à part entière, à reprendre le moment venu.
- **Aide à l'installation sur iPhone** : Safari ne propose jamais l'invite d'installation automatique ; un bouton affichant les instructions « Partager, puis Sur l'écran d'accueil » serait utile.
- **Webhook GitHub** pour un déploiement instantané, comme exercice d'intégration continue, au prix d'un point d'entrée supplémentaire à sécuriser.

Annexe — Récapitulatif des commandes

```
# --- Étape A : déposer Bookaru sur le NAS ---
APPDATA=/srv/dev-disk-by-uuid-33ff53ec-a8f7-4e25-8c31-4cf4f9c452de/appdata
mkdir -p $APPDATA/bibliotrack
cd $APPDATA/bibliotrack
git clone https://github.com/Relyzim/app_biblio_nini.git repo
nano $APPDATA/bibliotrack/Caddyfile          # contenu : section 3.4
# puis stack Portainer « bibliotrack » (section 3.5)
# test : http://192.168.1.134:8095

# --- Étape B : Internet ---
# domaine bookaru.app (Cloudflare Registrar), Zero Trust (Free),
# tunnel « bookaru », stack Portainer « cloudflared » (variable CF_TUNNEL_TOKEN),
# route : bookaru.app -> http://192.168.1.134:8095
# test : https://bookaru.app

# --- Étape C : durcissement ---
docker network create bookaru-net
# réseau ajouté aux deux stacks, route -> http://bibliotrack:80,
# puis retrait de « ports: 8095:80 » de la stack Caddy
docker network inspect bookaru-net --format '{{range .Containers}}{{.Name}} {{end}}'

# --- Étape D : mises à jour ---
cd $APPDATA/bibliotrack/repo && git pull     # manuel
chmod +x $APPDATA/bibliotrack/auto-pull.sh   # automatique (tâche planifiée */3)
```